```
import seaborn as sns
import pandas as pd
df=sns.load_dataset("titanic")
df
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	de
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	N
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	N
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	N
...	...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	N
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	N
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	N

891 rows × 15 columns

Next steps: Generate code with df New interactive sheet

```
x=df.drop ("survived",axis=1)
y=df["survived"]
```

```
x = df.drop(['class', 'who', 'adult_male', 'deck', 'embark_town', 'alive', 'alone', 'survived'],
x['age'].fillna(x['age'].median(), inplace=True)
x = pd.get_dummies(x, columns=['sex', 'embarked'], drop_first=True)
```

```
/tmp/ipykernel_351/1368210512.py:2: FutureWarning: A value is trying to be set on a copy of a Dat
The behavior will change in pandas 3.0. This inplace method will never work because the intermedi

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},


  x['age'].fillna(x['age'].median(), inplace=True)
```

```
y = df["survived"]
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression(max_iter=1000)
```

```
log_model.fit(x_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

```
y_pred = log_model.predict(x_test)

y_prob = log_model.predict_proba(x_test)
```

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8100558659217877
```

```
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print(conf_matrix)
```

```
[[90 15]
 [19 55]]
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

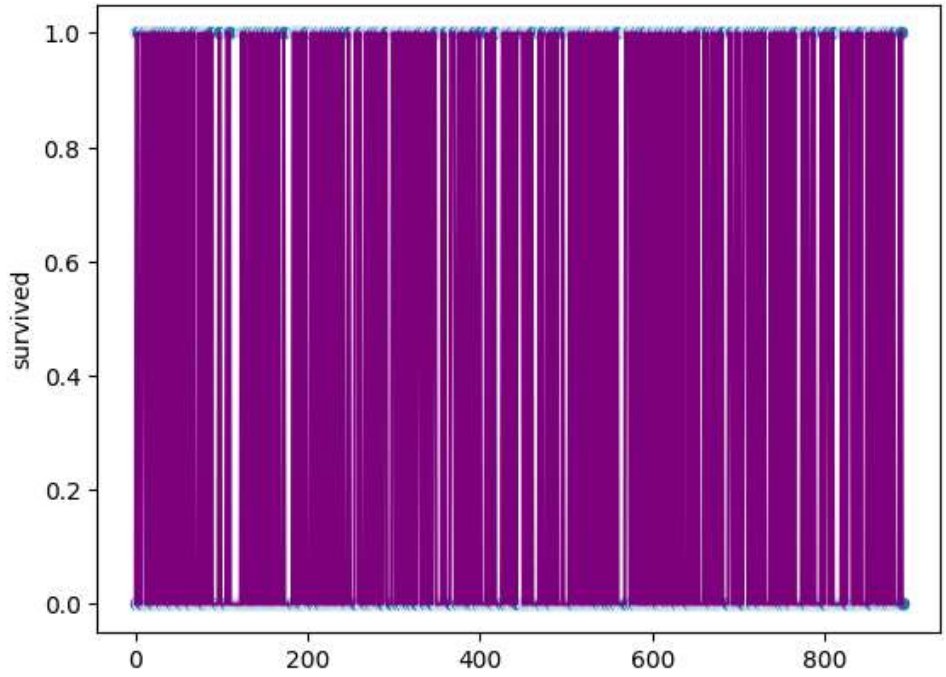
```
              precision    recall  f1-score   support

           0       0.83      0.86      0.84       105
           1       0.79      0.74      0.76        74

    accuracy                           0.81       179
   macro avg       0.81      0.80      0.80       179
weighted avg       0.81      0.81      0.81       179
```

```
sns.scatterplot(y)
sns.lineplot(y, color='purple')
```

```
<Axes: ylabel='survived'>
```



svm

```
from sklearn.svm import SVC
```

```
model= SVC(kernel='rbf', C=10, gamma='scale')
model.fit(x_train,y_train)
```

```
SVC
SVC(C=10)
```

```
y_pred1 = model.predict(x_test)

# y_prob1 = model.predict_proba(x_test)
```

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred1)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.7206703910614525
```

```
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_test, y_pred1)
print(conf_matrix)
```

```
[[98  7]
 [43 31]]
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred1))
```

```
              precision    recall  f1-score   support

           0       0.70      0.93      0.80       105
           1       0.82      0.42      0.55        74

    accuracy                           0.72       179
   macro avg       0.76      0.68      0.68       179
weighted avg       0.74      0.72      0.70       179
```